

TaskLearning

Plataforma de Estudio Asistido por IA

CUJAE — Ingeniería Informática — 2do Año

Documento de arquitectura y plan de implementación.

Versión: 3.1 — Septiembre 2026

Estado actual: Frontend funcional (vanilla JS + pdf.js)

Objetivo: Sistema completo con backend, IA real y RAG.

Contenido

- 1. Estado actual del proyecto
- 2. Arquitectura propuesta
- 3. Stack tecnológico
- 4. Modelo de datos
- 5. API REST — Endpoints
- 6. Pipeline de procesamiento IA
- 7. AI Service — Abstracción
- 8. RAG — Retrieval Augmented Generation
- 9. Tutor IA
- 10. Plan de implementación por etapas
- 11. Variables de entorno
- 12. Dependencias

1. Estado Actual

El proyecto es una aplicación web 100% estática desplegada en GitHub Pages.

No tiene backend, base de datos real, ni integración con IA.

Componentes existentes

- Frontend: SPA vanilla JS + CSS con tema púrpura oscuro
- Extracción de PDF: pdf.js v3.11 (en cliente)
- Análisis: Keyword matching y regex (no es IA real)
- Persistencia: localStorage del navegador
- Hosting: GitHub Pages (estático)

Funcionalidades que funcionan

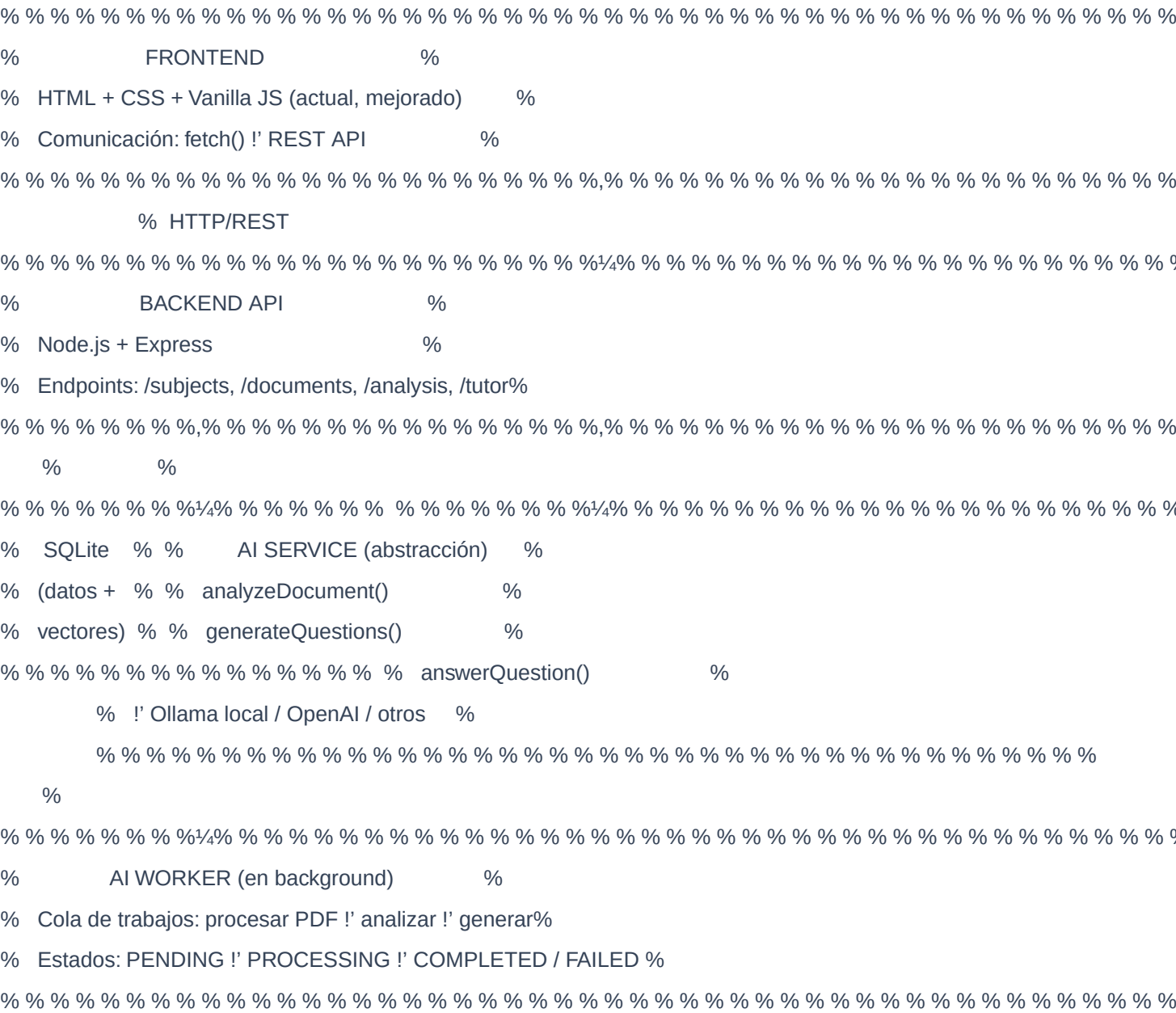
- Drag & drop de PDFs
- Extracción de texto del PDF
- Detección de temas por patrones
- Generación de tests interactivos (con scores)
- Generación de ejercicios y repasos
- Recomendación de IA (Kimi/ChatGPT/Claude)
- Búsqueda y filtros de asignaturas
- UI profesional con tabs de detalle

Qué falta

- Backend API (no existe)
- Base de datos real (solo localStorage)
- IA real (solo keyword matching)
- Procesamiento en segundo plano
- Sistema RAG para tutor IA
- Autenticación de usuarios

2. Arquitectura Propuesta

Arquitectura de capas con separación clara entre frontend, backend, IA y datos.



3. Stack Tecnológico

Backend

- Runtime: Node.js
- Framework: Express.js
- Base de datos: SQLite (better-sqlite3)
- Extracción PDF: pdf-parse
- Subida de archivos: multer
- IDs únicos: uuid
- Variables de entorno: dotenv

IA

- Proveedor: Ollama local (primero), luego OpenAI
- Modelo configurable via variable de entorno
- Abstracción AIService para cambiar proveedor sin reescribir

Frontend

- Se mantiene vanilla JS + CSS actual
- Comunicación con backend via fetch() API
- Polling para estado de procesamiento

Futuro (no ahora)

- Redis + BullMQ para cola de jobs
- pgvector para embeddings vectoriales
- WebSocket para tiempo real

4. Modelo de Datos

subjects

- id (TEXT PRIMARY KEY — UUID)
- name (TEXT NOT NULL)
- description (TEXT)
- created_at (TEXT)
- updated_at (TEXT)

documents

- id (TEXT PRIMARY KEY)
- subject_id (FK !' subjects)
- name (TEXT)
- type (TEXT — plan_temático|libro|conferencia|ejercicio|otro)
- file_path (TEXT — ruta del archivo en disco)
- status (TEXT — PENDING|PROCESSING|COMPLETED|FAILED)
- page_count (INTEGER)
- chunk_count (INTEGER)
- error_message (TEXT)
- processed_at (TEXT)
- created_at (TEXT)
- metadata (TEXT — JSON)

chunks (para RAG futuro)

- id (TEXT PRIMARY KEY)
- document_id (FK !' documents)
- page_number (INTEGER)
- chunk_index (INTEGER)
- content (TEXT — el texto del chunk)
- token_count (INTEGER)

units

- id, subject_id (FK), name, order, description

topics

- id, unit_id (FK), name, order, content_summary, objectives (JSON)

concepts

- id, topic_id (FK), name, definition, source_ref (JSON)

summaries

- id, topic_id (FK), content, source_refs (JSON), generated_at

questions

- id, topic_id (FK), type, question_text, options (JSON), correct_answer, explanation, source_refs

flashcards

- id, topic_id (FK), front, back, difficulty, source_refs

exercises

- id, topic_id (FK), statement, solution, type, source_refs

ai_jobs

- id, subject_id, document_id, type, status (PENDING|PROCESSING|COMPLETED|FAILED)
- input_data (JSON), output_data (JSON), error_message
- started_at, completed_at, created_at

5. API REST — Endpoints

Asignaturas

- GET /api/subjects — Listar todas
- POST /api/subjects — Crear asignatura
- GET /api/subjects/:id — Detalle de asignatura
- PUT /api/subjects/:id — Actualizar asignatura
- DELETE /api/subjects/:id — Eliminar asignatura

Documentos

- POST /api/documents — Subir PDF (multipart/form-data)
- GET /api/documents?subject_id=X — Listar docs de una asignatura
- GET /api/documents/:id — Estado y metadatos de un doc
- DELETE /api/documents/:id — Eliminar documento

Análisis

- POST /api/analysis/process/:docId — Iniciar procesamiento de un doc
- GET /api/analysis/status/:docId — Consultar estado del procesamiento
- GET /api/analysis/units/:subjectId — Unidades generadas
- GET /api/analysis/topics/:unitId — Temas de una unidad

Materiales generados

- GET /api/materials/summaries/:topicId
- GET /api/materials/questions/:topicId
- GET /api/materials/flashcards/:topicId
- GET /api/materials/exercises/:topicId
- GET /api/materials/reviews/:topicId

Tutor IA

- POST /api/tutor/ask — Preguntar al tutor
- Body: { subject_id, question }
- Respuesta: { answer, sources: [{doc, page, chunk}] }

Tests

- POST /api/tests/generate/:topicId — Generar test para un tema
- POST /api/tests/verify — Verificar respuestas

6. Pipeline de Procesamiento IA

Flujo cuando el usuario sube un PDF:

1. Frontend envía PDF via POST /api/documents
2. Backend guarda el archivo y crea registro con status=PENDING
3. Backend crea ai_job type=process_document, status=PENDING
4. Worker detecta el job y lo toma
5. Worker actualiza status=PROCESSING
6. Worker extrae texto del PDF (pdf-parse)
7. Worker divide en chunks (~500 tokens c/u)
8. Worker guarda chunks en DB
9. Worker envía chunks al AIService para análisis
10. AIService identifica: unidades, temas, conceptos, definiciones
11. Worker guarda estructura en DB (units, topics, concepts)
12. Worker genera resúmenes por tema via AIService
13. Worker guarda summaries en DB
14. Worker genera preguntas, flashcards, ejercicios via AIService
15. Worker guarda materials en DB
16. Worker actualiza status=COMPLETED, document.status=COMPLETED
17. Frontend detecta cambio de estado y muestra resultados

7. AI Service — Abstracción

Interfaz común para cambiar proveedor sin modificar el backend.

Métodos

- `analyzeDocument(chunks) !' { units, topics, concepts }`
- `generateSummary(topic, context) !' string`
- `generateQuestions(topic, context, count) !' Question[]`
- `generateFlashcards(topic, context) !' Flashcard[]`
- `generateExercise(topic, context) !' Exercise`
- `generateReview(topic, context) !' string`
- `answerQuestion(question, context) !' { answer, sources }`

Proveedores soportados

- Ollama local (`http://localhost:11434/v1`) — GRATIS, PRIVADO
- OpenAI API — Pago, mejores modelos
- Cualquier compatible con OpenAI SDK

Configuración

- `AI_PROVIDER=ollama | openai | other`
- `AI_MODEL=gemma4:26b | gpt-4o-mini | ...`
- `AI_API_KEY=sk-...` (vacío para Ollama)
- `AI_BASE_URL=http://localhost:11434/v1` (para Ollama)

8. RAG — Retrieval Augmented Generation

Retrieval Augmented Generation: buscar fragmentos relevantes antes de generar una respuesta.

Flujo RAG

1. Usuario hace una pregunta
2. Se genera un embedding de la pregunta
3. Se busca en la DB los chunks más similares (similitud coseno)
4. Se recuperan los top-K chunks más relevantes
5. Se construye un prompt con: pregunta + contexto recuperado
6. El modelo genera una respuesta fundamentada en el contexto
7. Se muestran las fuentes: "Basado en: Material X, págs 14-16"

Implementación

- Embeddings: Ollama (nomic-embed-text) o OpenAI (text-embedding-3-small)
- Almacenamiento: SQLite con extensión vectorial, o tabla de similitud simple
- Búsqueda: Para PoC, búsqueda por keywords + TF-IDF. Luego vectores.
- K=5 fragmentos por defecto, configurable

9. Tutor IA

El tutor IA permite al estudiante hacer preguntas sobre la asignatura.

Ejemplos de uso

- "Explícame el tema 3 de la asignatura"
- "¿Qué significa el concepto de polimorfismo?"
- "Créame un examen de la unidad 2"
- "¿Qué temas debería repasar para el parcial?"
- "Explícame esto como si fuera principiante"
- "¿Qué relación existe entre estos dos conceptos?"
- "Busca en mis materiales dónde se explica X"

Flujo

1. Frontend envía POST /api/tutor/ask { subject_id, question }
2. Backend busca chunks relevantes via RAG
3. Backend construye prompt con contexto recuperado
4. AIService genera respuesta fundamentada
5. Backend retorna { answer, sources }
6. Frontend muestra respuesta + referencias

10. Plan de Implementación

ETAPA 1 — Backend + DB + Upload

- Inicializar Node.js + Express
- Configurar SQLite + migraciones
- Endpoints: CRUD subjects, upload documents
- Adaptar frontend para enviar PDFs al backend

ETAPA 2 — Extracción + Chunking

- Instalar pdf-parse
- Worker de procesamiento de PDF
- Extracción de texto por página
- División en chunks (~500 tokens)
- Guardar chunks en DB

ETAPA 3 — AI Service + Análisis

- Crear AIService con abstracción
- Configurar Ollama local
- Pipeline: chunks → unidades → temas → conceptos
- Generar resúmenes por tema

ETAPA 4 — Generación de Materiales

- Generar preguntas (multiple choice, true/false)
- Generar flashcards (frente/atras)
- Generar ejercicios con soluciones
- Generar repasos con tips de estudio
- Todo con referencias a fuentes

ETAPA 5 — Embeddings + RAG

- Generar embeddings de chunks
- Búsqueda semántica
- Preparar contexto para tutor

ETAPA 6 — Tutor IA

- Endpoint POST /api/tutor/ask
- Chat interactivo con materiales
- Historial de conversación

ETAPA 7 — Tests con IA real

- Reemplazar tests generados por regex
- Preguntas de calidad con IA
- Scores en DB

ETAPA 8 — Sistema Adaptativo (futuro)

- Detectar temas débiles
- Ejercicios focalizados
- Planes de estudio personalizados

11. Variables de Entorno

Backend

- PORT=3000
- DATABASE_URL=./data/tasklearning.db
- UPLOAD_DIR=./uploads
- MAX_FILE_SIZE=52428800

AI

- AI_PROVIDER=ollama
- AI_MODEL=gemma4:26b
- AI_API_KEY=
- AI_BASE_URL=http://localhost:11434/v1

Frontend (para conectar con backend)

- API_URL=http://localhost:3000/api

12. Dependencias

Backend (package.json)

- express — web framework
- better-sqlite3 — SQLite driver
- pdf-parse — extracción de texto PDF
- multer — subida de archivos
- uuid — IDs únicos
- dotenv — variables de entorno
- cors — CORS para frontend
- openai — SDK compatible con Ollama

Frontend

- Ninguna dependencia nueva — se mantiene vanilla JS

Documento generado automáticamente — TaskLearning v3.1

Septiembre 2026